Competitive Programming

Section 44: String Algorithms II

Competitive Programming Course

2026-10-24

Table of contents

1 Three structures, three questions **3**

2 Tries: prefixes of many words **4**
2.1 Trie operations and implementation notes 5
2.2 Reusable C++ trie 5
2.3 Reusable Python trie 6

3 Palindrome queries **7**
3.1 Rolling hash: reusable implementation 8

4 Suffix automata: from substring groups to suffix links and clones **12**
4.1 The starting problem: compress all substrings 12
4.2 A first complete automaton: `aba` 13
4.3 What is a state? The meaning of `endpos` 14
4.4 Define the suffix link by shortening a string from the left 19
4.5 The two kinds of arrows 20
4.6 How `extend(ch)` builds the automaton 21
4.7 Understand the clone: `ab` becomes `abb` 23
4.8 The complete `abcbc` automaton and its suffix-link tree 28
4.9 Commented C++ and Python implementations 31
4.10 Quick reference and checks 35

5 Implementation notes and tests **36**

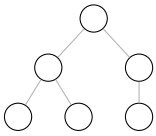
6 Exercises **37**

1 Three structures, three questions

Section 43 had one object, the prefix function, and everything came out of it. This section is the opposite: three unrelated structures, and the skill being taught is choosing between them. Picking the wrong one is not a slow solution, it is usually an impossible one.

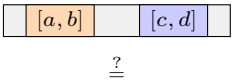
The question to ask first is never "which structure do I know?" but **what object is the query about?**

Trie
many strings,
shared prefixes



how many words start with ap?
walk $|p|$ edges

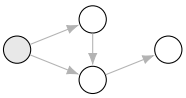
Rolling hash
one string,
compare two windows



$\stackrel{?}{=}$

are these two windows equal?
$O(1)$ after $O(n)$ prep,
probabilistic

Suffix automaton
one string,
every substring at once



how many distinct substrings?
$O(n)$ states
each groups several substrings
count by length (Chapter 4)

They are not interchangeable, and that is the whole skill here. A suffix automaton answers nothing useful about a *dictionary* of separate words — it is built from one string. A trie cannot compare two arbitrary substrings of one long text — it indexes prefixes, not positions. Before writing code, say out loud which object the query names: a prefix of a dictionary word, a window inside one text, or the set of all substrings. The answer picks the structure.

In the automaton panel, **one circle can represent several distinct substrings**. Counting circles therefore does not give the answer. Each state stores a consecutive range of substring lengths, with one substring per length. The formula introduced in Chapter 4 counts the lengths in each state, then adds these counts. For example, in `aba`, one state represents both `ba` and `aba`: that single state contributes two substrings.

The query names	Reach for	Section problem
a prefix of a dictionary word	trie	A, D
a window inside one text	rolling hash, or direct comparison	B
the set of all substrings of one text	suffix automaton or suffix array	C

Each of the three chapters below takes one row of that table.

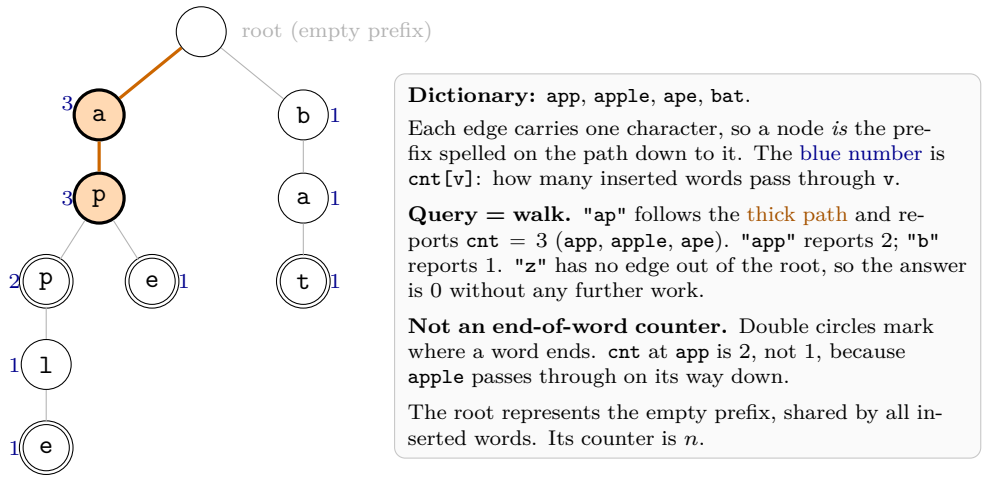
2 Tries: prefixes of many words

A trie is a rooted tree whose edges carry single characters, so a node *is* the prefix spelled along the path from the root to it. Shared prefixes are stored once, which is the entire point.

To answer "how many dictionary words begin with `p`", store on each node the number of inserted words whose path passes through it:

```
insert(word):
    node = root
    cnt[node] += 1
    for c in word:
        create edge c if missing
        node = child[node][c]
        cnt[node] += 1

query(p):
    node = root
    for c in p:
        if edge c missing: return 0
        node = child[node][c]
    return cnt[node]
```



Two details decide whether this is correct.

`cnt` is not an end-of-word counter. It counts words *passing through*, so a node can hold a count larger than the number of words ending there. In the figure, `app` has `cnt = 2` because `apple` continues below it. If you need to know how many words *end* at a node, that is a second, separate counter.

The root counts the empty prefix. Increment it once per inserted word, so `cnt[root]` equals the number of dictionary words. Each move to a child then increments the counter for

the corresponding nonempty prefix. A missing edge ends a query immediately with `0`, with no need to walk the rest of the pattern.

2.1 Trie operations and implementation notes

The reusable `Trie` below provides `insert(word)` and `count_prefix(prefix)`. Both take time proportional to the argument length. The dictionary is restricted to lowercase English letters. Duplicate insertions count separately. Node 0 is the root; child index 0 means an absent edge because all allocated children have positive indices. `pass` counts all words visiting a node, including words that continue below it. This implementation also increments the root, so querying the empty prefix returns the number of words.

In C++, store indices rather than references to nodes: `push_back` can move the vector's storage. In Python, create a fresh dictionary for each new node. Memory is `O(L)` for fixed alphabet size, where `L` is total inserted length.

2.2 Reusable C++ trie

```cpp
#pragma once

#include <bits/stdc++.h>
using namespace std;

class Trie {
    struct Node {
        array<int, 26> next{};
        int pass = 0;
    };
    vector<Node> nodes{Node{}};

public:
    void insert(const string& word) {
        int v = 0;
        nodes[v].pass++;
        for (char ch : word) {
            int c = ch - 'a';
            if (nodes[v].next[c] == 0) {
                int child = nodes.size();
                nodes[v].next[c] = child;
                nodes.push_back(Node{});
            }
            v = nodes[v].next[c];
            nodes[v].pass++;
        }
    }

    int count_prefix(const string& prefix) const {
        int v = 0;
```

```
        for (char ch : prefix) {
            int child = nodes[v].next[ch - 'a'];
            if (child == 0) {
                return 0;
            }
            v = child;
        }
        return nodes[v].pass;
    }
};
```

2.3 Reusable Python trie

```
class Trie:
    def __init__(self):
        self.children = [{}]
        self.passed = [0]

    def insert(self, word):
        v = 0
        self.passed[v] += 1
        for ch in word:
            if ch not in self.children[v]:
                child = len(self.children)
                self.children[v][ch] = child
                self.children.append({})
                self.passed.append(0)
            v = self.children[v][ch]
            self.passed[v] += 1

    def count_prefix(self, prefix):
        v = 0
        for ch in prefix:
            if ch not in self.children[v]:
                return 0
            v = self.children[v][ch]
        return self.passed[v]
```

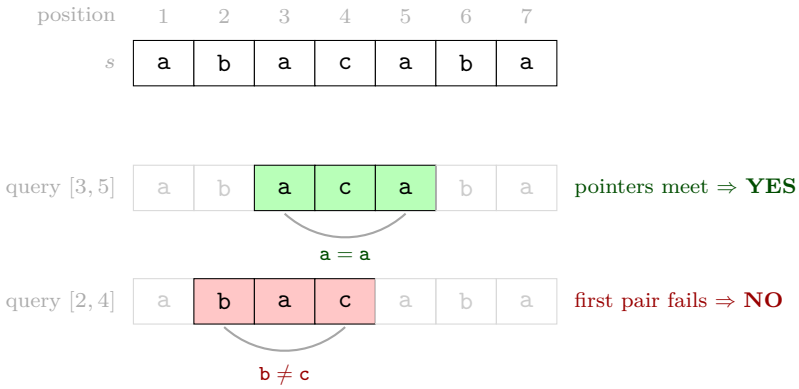
3 Palindrome queries

We now leave the dictionary setting. There is only one text, and every query selects a consecutive part of it. The task is to decide whether that selected part reads the same from left to right and from right to left. Such a string is called a **palindrome**: `a`, `aba`, and `abba` are palindromes; `abc` is not.

For example, keep the text `abacaba` fixed. A query on positions 3 through 5 selects `aca`, so the answer is YES. A query on positions 2 through 4 selects `bac`, so the answer is NO. The queries do not change the text.

Start with the condition itself. For a string to equal its reverse, its first and last characters must match, then its second and next-to-last characters, and so on. This gives a direct algorithm: place one pointer at each end and move them inward. A mismatching pair proves NO immediately. If the pointers meet or cross without a mismatch, every required pair matched, so answer YES. An odd-length palindrome has a middle character that needs no comparison.

The figure below follows these comparisons on two selected parts of the same text. Only the characters inside the selected part participate in the test:



Compare symmetric pairs. In `aca`, the two outer `a` characters match; the middle `c` needs no comparison. In `bac`, the first pair already differs, so the answer is NO. An even-length palindrome finishes when the pointers cross; an odd-length palindrome finishes when they meet.

Cost of the direct scan. A selected part of length k requires at most $\lfloor k/2 \rfloor$ comparisons. This is exact and costs $O(k)$. Many long queries can repeat the same work, which motivates the preprocessing introduced below.

```
while l < r:
    if s[l] != s[r]: return NO
    l += 1
    r -= 1
return YES
```

This loop tests one selected substring. If that substring has `k` characters, the loop compares at most `floor(k / 2)` pairs, so the test takes `O(k)` time and `O(1)` extra memory. It returns YES precisely when every symmetric pair matches; a single mismatch is enough to return NO.

For example, testing `aca` needs one comparison (`a` against `a`). Testing `abccba` needs three comparisons: `a` against `a`, then `b` against `b`, then `c` against `c`.

Now suppose we must answer `q` queries on a text of length `n`. Running this loop separately for every query can take `O(n*q)` time if all selected ranges are long. The same characters may be compared repeatedly. This repeated work motivates preprocessing the unchanged text so that later queries can reuse the results.

A **rolling hash** assigns a numerical fingerprint to a string. Prefix fingerprints and precomputed powers let us extract the fingerprint of any substring in `O(1)` time. Equal strings always have equal fingerprints; different strings can occasionally share one, which is called a collision. This tradeoff is why hashing is fast but probabilistic.

A palindrome is a substring equal to its reverse. Build fingerprints for the text and for its reversed copy. A selected part of the original text has a corresponding part in the reversed text, with its characters in reverse order. Compare the two fingerprints.

For example, selecting `bac` in the original text corresponds to `cab` in the reversed text. Their fingerprints differ, so this is not a palindrome. Selecting `aca` corresponds to `aca`; their fingerprints match, as expected.

The next subsection derives the fingerprint formula and gives reusable C++ and Python implementations. Endpoint conventions are specified with the range-hash operation, where they are needed to use the code.

Hashing is `O(1)` per query after `O(n)` preprocessing, but it is probabilistic. Manacher's algorithm and palindromic trees are the deterministic alternatives. When the constraints permit the direct check, prefer it.

Problem B allows `n, q <= 200000`. Scanning every selected range could therefore examine about `40000000000` characters. A direct scan is a useful small-input oracle, but the exercise requires preprocessing so that long queries can be answered without reading all their characters again.

3.1 Rolling hash: reusable implementation

The direct scan can inspect `O(n)` characters per query, for `O(n*q)` total time. Because the text never changes, we can preprocess it and reuse the results instead of scanning each selected range again.

The idea is to replace a string by a numerical fingerprint. Interpret its characters as digits in base `BASE`: with `a = 1`, `b = 2`, and `c = 3`, the fingerprint of `abc` before taking a remainder is:

```
1 * BASE^2 + 2 * BASE + 3
```

Appending a character multiplies the old fingerprint by `BASE` and adds the new character's value. Taking a remainder modulo `MOD` at each step keeps numbers bounded. This is a rolling hash. Equal strings have the same hash, but different strings can collide after taking the remainder.

Store the hash of every prefix in `h`, and store powers of the base in `power`. To extract a substring, subtract the preceding prefix shifted by the substring length. For instance, the hash of `bc` is obtained from `abc` by subtracting `hash(a) * BASE^2`, leaving `2 * BASE + 3`. The same subtraction works modulo `MOD`, so we do not need to reconstruct the large integer.

One hash can collide: two different strings can produce the same remainder. To make collisions harder, the implementation computes the hash twice, using the same base but two different divisors:

- `MOD1 = 1000000007` for the first hash.
- `MOD2 = 1000000009` for the second hash.

The result for a substring is a pair `(first_hash, second_hash)`. To compare two substrings of the same length, compare both numbers. If either number differs, the substrings are different. If both match, the substrings may be equal, but a collision in both calculations is still possible.

The C++ and Python classes implement this same pair of calculations. In each calculation, `MOD` below means the chosen divisor (`MOD1` or `MOD2`); keep separate prefix-hash and power arrays for the two divisors. Initialize `h[0] = 0`, `power[0] = 1`, then use:

```
h[i + 1] = (h[i] * BASE + value(s[i])) % MOD
power[i + 1] = power[i] * BASE % MOD
hash(l, r) = (h[r] - h[l] * power[r - l]) % MOD
```

Here `value('a') = 1`, …, `value('z') = 26`. `get(l, r)` uses **0-based, half-open** endpoints: it hashes `s[l:r]`, including `l` and excluding `r`. The empty range has hash `(0, 0)`. Compare ranges of equal length, built with the same parameters. Subtracting the preceding prefix normalizes the hash, so identical substrings at different positions have identical hashes.

Construction takes `O(n)` time and memory; each `get` takes `O(1)` time. Unequal hash pairs prove inequality. Equal pairs can collide, so even two moduli do not turn this into an exact equality test. The fixed constants make the example reproducible; for adversarial inputs, randomizing a shared base is an additional precaution. Use the direct scan described above when exactness is required and the query lengths allow it.

3.1.1 Reusable C++ rolling hash

Construct `RollingHash` from a lowercase string. `get(l, r)` returns the pair of hash residues for the 0-based half-open range `[l,r)`. The class stores its prefix hashes and powers; no query loop or input format is built into it.

```cpp
#pragma once

#include <bits/stdc++.h>
using namespace std;

class RollingHash {
    static constexpr long long MOD1 = 1000000007;
    static constexpr long long MOD2 = 1000000009;
    static constexpr long long BASE = 911382323;
    vector<long long> h1, h2, p1, p2;

public:
    explicit RollingHash(const string& s) {
        int n = s.size();
        h1.assign(n + 1, 0);
        h2.assign(n + 1, 0);
```

```
        p1.assign(n + 1, 1);
        p2.assign(n + 1, 1);
        for (int i = 0; i < n; i++) {
            int value = s[i] - 'a' + 1;
            h1[i + 1] = (h1[i] * BASE + value) % MOD1;
            h2[i + 1] = (h2[i] * BASE + value) % MOD2;
            p1[i + 1] = p1[i] * BASE % MOD1;
            p2[i + 1] = p2[i] * BASE % MOD2;
        }
    }

    // Hash of s[l:r], with 0-based, half-open endpoints.
    pair<long long, long long> get(int l, int r) const {
        long long a = (h1[r] - h1[l] * p1[r - l]) % MOD1;
        long long b = (h2[r] - h2[l] * p2[r - l]) % MOD2;
        if (a < 0) {
            a += MOD1;
        }
        if (b < 0) {
            b += MOD2;
        }
        return {a, b};
    }
};
```

C++ products use `long long`: both residues and the base are below `10^9 + 9`, so their products fit in a signed 64-bit integer. Normalize negative remainders before returning the hash.

3.1.2 Reusable Python rolling hash

The Python class offers the same constructor and `get(l, r)` operation. Python's `%` already produces a nonnegative residue for positive moduli.

```
class RollingHash:
    MOD1 = 1_000_000_007
    MOD2 = 1_000_000_009
    BASE = 911_382_323

    def __init__(self, s):
        n = len(s)
        self.h1 = [0] * (n + 1)
        self.h2 = [0] * (n + 1)
        self.p1 = [1] * (n + 1)
        self.p2 = [1] * (n + 1)
        for i, ch in enumerate(s):
            value = ord(ch) - ord("a") + 1
            self.h1[i + 1] = (
                self.h1[i] * self.BASE + value
            ) % self.MOD1
            self.h2[i + 1] = (
```

```
                self.h2[i] * self.BASE + value
            ) % self.MOD2
            self.p1[i + 1] = self.p1[i] * self.BASE % self.MOD1
            self.p2[i + 1] = self.p2[i] * self.BASE % self.MOD2

    def get(self, l, r):
        # Hash of s[l:r], with 0-based, half-open endpoints.
        a = (
            self.h1[r] - self.h1[l] * self.p1[r - l]
        ) % self.MOD1
        b = (
            self.h2[r] - self.h2[l] * self.p2[r - l]
        ) % self.MOD2
        return a, b
```

Both versions take `O(n)` time and memory to build and `O(1)` time per range hash. Comparing ranges against a reversed text is an application of this general substring-hash API; its equality test remains probabilistic.

4 Suffix automata: from substring groups to suffix links and clones

4.1 The starting problem: compress all substrings

Imagine a text `s` of length `n = 100000`. It has `n * (n + 1) / 2 = 5000050000` nonempty substring occurrences: about five billion. The number of *different* substring values can also be quadratic. A trie containing every substring can therefore require `O(n^2)` nodes in the worst case, which is far too much memory at this scale.

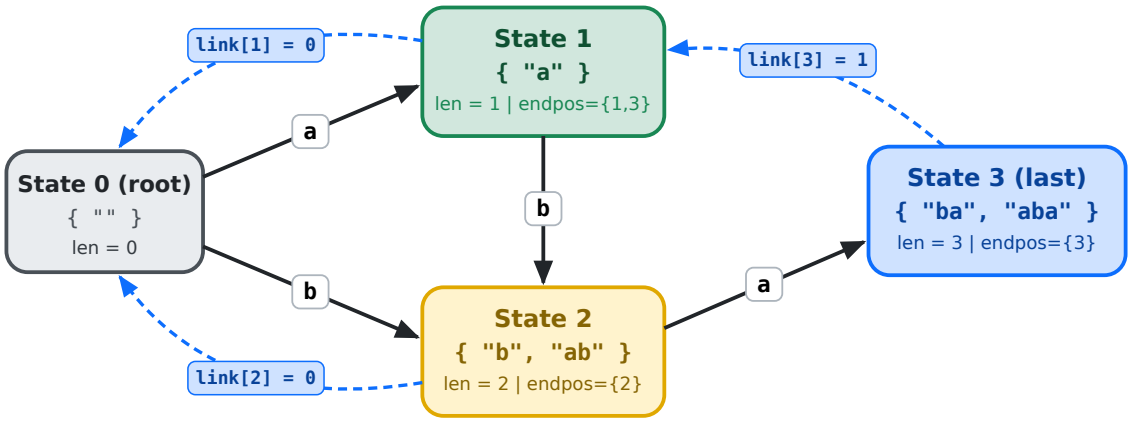
A **suffix automaton**, or **SAM**, represents all those substrings using `O(n)` states and transitions. It can be built one character at a time in `O(n)` time for a fixed alphabet with constant-time transitions. More precise bounds are `2*n - 1` states for `n >= 2` and `3*n - 4` character transitions for `n >= 3`. The empty text has one root; a one-character text has two states. The C++ implementation below uses maps, adding a `log alphabet` factor.

The central idea: group strings with the same ending positions. A trie stores a separate node for each different prefix on its paths. A SAM merges substrings that finish at exactly the same positions in one text. It indexes all substrings, including ones that start in the middle, such as `ba` in `aba`.

Strings grouped in one state share their right-hand end: shorter strings are **suffixes** of longer ones, not prefixes. For example, one state can group `cbc`, `bcbc`, and `abcbc`. We will make this grouping concrete before studying the construction.

4.2 A first complete automaton: `aba`

Start with the complete automaton for `s = "aba"`. Solid black arrows are character transitions; dashed blue arrows are suffix links, stored in `link`. The boxes show the strings represented by each state, rather than just its numeric identifier.



Complete automaton for `aba`. Black: character transitions; blue dashed: suffix links.

Read every pattern from state 0, following character transitions.

Pattern	Route from the root	State reached
`a`	$0 \xrightarrow{a} 1$	1
`b`	$0 \xrightarrow{b} 2$	2
`ab`	$0 \xrightarrow{a} 1 \xrightarrow{b} 2$	2 again
`ba`	$0 \xrightarrow{b} 2 \xrightarrow{a} 3$	3
`aba`	$0 \xrightarrow{a} 1 \xrightarrow{b} 2 \xrightarrow{a} 3$	3 again

Notice that `ba` is not a prefix of `aba`, but we can still read it directly from state 0. If an arrow is missing, the pattern is absent. For example, `aa` reaches state 1 after its first character, then fails because state 1 has no `a` arrow. Finishing the pattern is enough: we may stop at any state.

4.2.1 Why is it called a suffix automaton if it finds all substrings?

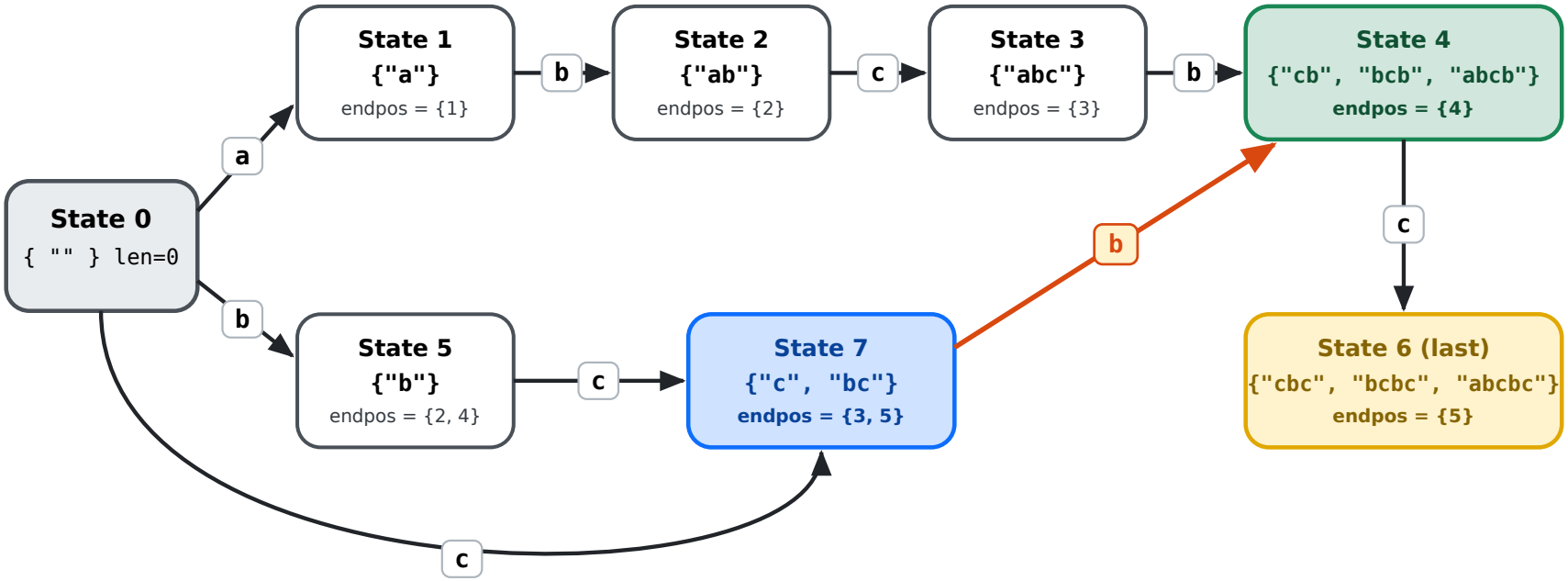
Every substring is a prefix of a suffix of the text, or equivalently a suffix of a prefix. Reading from the root and allowing a stop at any state therefore lets us recognize all substrings. To recognize only suffixes, we would mark the states reached by complete suffixes as accepting. In this chapter, `contains` accepts a pattern whenever its entire path exists.

The suffix structure also appears *inside* a state: `ba` is a suffix of `aba`, so those strings can share a state. They are not two prefixes being grouped as in a dictionary trie.

4.3 What is a state? The meaning of `endpos`

Use the richer text `abcbc`, with positions numbered 1 through 5. For each substring, record all positions where an occurrence ends: this set is called `endpos`. The diagram shows the states' groups and ending positions. State identifiers are not text positions.

Character transitions and ending-position groups for `abcbc`.



The indexed text is:

Position	1	2	3	4	5
Character	a	b	c	b	c

Focus on the substrings ending in c. They reach either state 6 or state 7:

Substring	Length	endpos	What can follow now?	State
abcbc	5	{5}	Nothing	6
bcbc	4	{5}	Nothing	6
cbc	3	{5}	Nothing	6
bc	2	{3, 5}	b, after position 3	7
c	1	{3, 5}	b, after position 3	7

4.3.1 Why can those strings share a state?

First consider state 6, which groups abcbc, bcbc, and cbc. All three finish only at position 5. They consequently have the same possible future: if we append x at position 6, all three can be extended by x. One state can represent the whole group {cbc, bcbc, abcbc}.

State 7 is more interesting because its strings occur twice. Why can bc and c still share a state?

String	First occurrence	Second occurrence	Ending positions
c	Position 3	Position 5	{3, 5}
bc	Positions 2-3	Positions 4-5	{3, 5}

Every c in this text has a b immediately before it. Whether we have read c or bc, the possible current ending positions are exactly the same: 3 and 5.

- At position 3, the next character is b. It extends c into cb and bc into bcb, both ending at position 4.
- At position 5, there is currently no next character. If we later append x, it will extend both c and bc at this position, into cx and bcx.

Same ending positions mean the same possible continuations. This remains true when there are several occurrences. A character that follows one of these strings at a particular ending position follows the other there too. That is why both can use the same outgoing transition table.

If we extend the text to abcbcbc, another bc ends at position 7. The strings c and bc still reach state 7, whose ending-position set becomes {3, 5, 7}. As long as every new c is preceded by b, they stay together. If a later c has no preceding b, as in abcbcac, their ending-position sets differ and a clone separates them.

4.3.2 Why does state 7 have a `b` arrow if the last `bc` has nothing after it?

A transition exists when at least one occurrence can continue with that character. It does not require every occurrence to continue.

Reading `bc` from the root reaches state 7, representing possible ending positions `{3, 5}`. Following `7 --b--> 4` then reads `bcb`:

- The occurrence ending at position 5 cannot continue with `b`; discard it.
- The occurrence ending at position 3 is followed by `b` at position 4; retain that occurrence and advance it.
- The destination is state 4, grouping `{cb, bcb, abcb}`, with `endpos = {4}`.

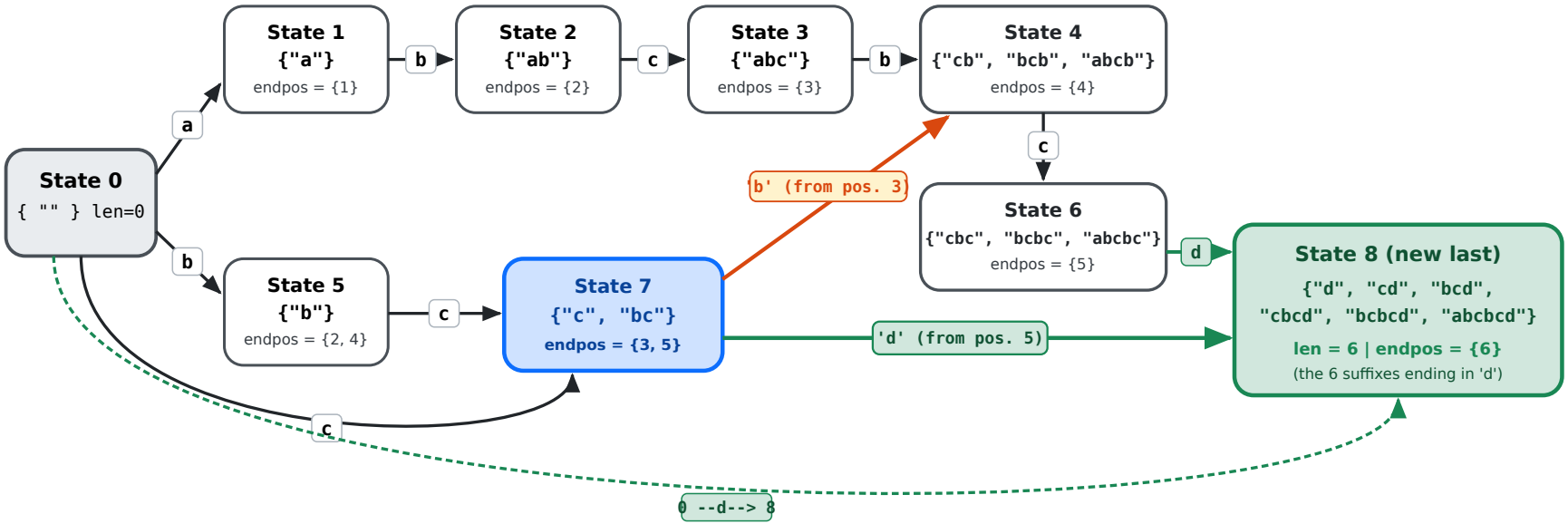
Following a character transition **filters and advances the possible ending positions**. Outgoing characters are the union of the characters following all occurrences. For state 7, position 3 contributes `{b}` and position 5 contributes no character, so their union is `{b}`. Since `c` and `bc` have the same ending positions, they have the same union and the same transitions.

4.3.3 Where is position 5 in state 7 if its outgoing path seems to use only position 3?

The final graph contains the route $7 \xrightarrow{b} 4 \xrightarrow{c} 6$. It is tempting to think that state 7 therefore means only position 3. But a state represents a set of possible occurrences, not one selected occurrence moving through the text.

1. **Stop after reading `bc`.** Starting at 0 and reading `b`, then `c`, stops in state 7, not state 6. The pattern occurs both in the middle, at positions 2-3, and at the end, at positions 4-5. Both ending positions are represented before we choose any further transition.
2. **The occurrence at the end adds no outgoing arrow yet.** Position 3 supplies the `b` arrow. Position 5 has nothing after it, so it contributes an empty set of next characters. An occurrence need not produce an arrow in order to belong to the state.
3. **Append `d` to make that last occurrence visible.** In `abcbcd`, the new character at position 6 follows the occurrence ending at position 5. Construction starts at state 6 and adds a `d` arrow to new state 8, for `cbcd`, `bcbcd`, and `abcbcd`. It then follows `link[6] = 7` and adds another `d` arrow to 8, for `cd` and `bcd`. That new arrow from state 7 comes from position 5, not position 3, whose following character is `b`. Finally, the root also receives a `d` arrow, for the one-letter string `d`.

Append d: the final occurrence represented by state 7 now supplies a continuation.



The new state's group is `{d, cd, bcd, cbcd, bcbcd, abcbcd}`. It contains the six suffixes ending in the new `d`, all with `endpos = {6}`.

4.3.4 The first invariant: what a state contains

A non-root state `v` is a box containing a group of substring values:

- All strings in the box are suffixes of one another, obtained by removing characters from the left of the longest string.
- Their lengths form one consecutive interval with no gaps, such as 3, 4, 5 in state 6, or 1, 2 in state 7.
- There is **exactly one string per represented length**. A shared ending position and a length uniquely determine the string.

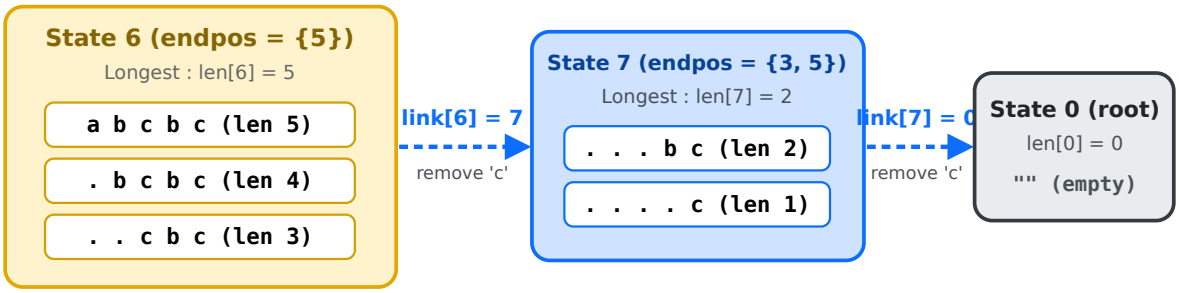
The drawn boxes list strings and ending positions to explain the invariant. The implementation does not store those lists or sets.

4.4 Define the suffix link by shortening a string from the left

A state does not store every string in its group. It stores `len[v]`, the length of its longest string, together with transitions and a suffix link. How can we find its shortest represented length, or move to shorter suffixes in another group? That is the role of `link[v]`.

4.4.1 Shorten `abcbc` one character at a time

Take the longest string in state 6, `abcbc`, of length 5. It ends only at position 5. Remove characters **from the left**, producing shorter suffixes:



Shorten `abcbc` from the left: the group changes when more ending positions appear.

While the suffix is `abcbc`, `bcbc`, or `cbc`, it occurs only at position 5. It stays in state 6. Shortening `cbc` to `bc` changes that:

- `bc` also occurs earlier, ending at position 3.
- Its ending-position set grows from `{5}` to `{3, 5}`.
- It therefore belongs to state 7 instead of state 6.

That first change of group is exactly `link[6] = 7`.

In general, start with the longest string represented by `v` and shorten it from the left. `link[v]` is the first different state reached, when the suffix becomes short enough to occur at additional ending positions. Equivalently, its longest string is the longest suffix that no longer belongs to `v`. If the first outside suffix is empty, the link points to the root.

This gives construction a shortcut between whole groups of shorter suffixes. When appending a character, it can follow this chain instead of rereading each suffix separately from state 0.

4.4.2 Count substrings without listing them

The suffix of length `len[link[v]]` is the first one outside state `v`. Therefore:

```
maximum represented length = len[v]
minimum represented length = len[link[v]] + 1
number of represented strings = len[v] - len[link[v]]
```

For state 6, `len[6] = 5` and `len[7] = 2`. Its lengths are 3, 4, and 5, so it contains `5 - 2 = 3` strings: `cbc`, `bcbc`, and `abcbc`.

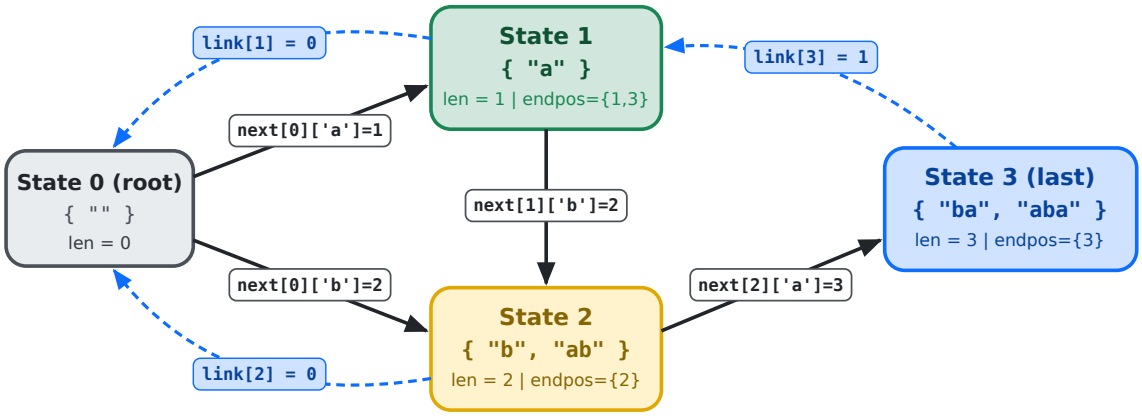
Every distinct nonempty substring belongs to exactly one non-root state. Adding the sizes of those groups gives:

$$\sum_{v \neq 0} (\texttt{len}[v] - \texttt{len}[\texttt{link}[v]]).$$

The lengths of different groups may overlap. For example, `a` and `b` both have length 1 but belong to different states. The groups of string values are disjoint, which is why the sum counts each string once.

4.5 The two kinds of arrows

The two kinds of arrows have different jobs. A character transition adds a character on the right; a suffix link removes characters from the left.



Character transitions extend on the right; suffix links shorten on the left.

Property	Character transition `next[v][ch]`	Suffix link `link[v]`
Effect on a string	Append `ch` on the right	Remove leading characters to reach a shorter group
Effect on maximum length	Increases `len`	Strictly decreases `len`
Graph structure	A directed acyclic graph reachable from 0	A tree rooted at 0, with links directed toward the parent
Use	Read a pattern starting at 0	Build the automaton and identify the represented length intervals

Only character transitions consume input. **A pattern lookup starts at 0 and never follows suffix links.** The empty root has `len[0] = 0` and `link[0] = -1`, where -1 is a sentinel rather than another state.

4.6 How `extend(ch)` builds the automaton

Suppose we have built the automaton for text `S`. The state containing the whole text is `last`. We now append a character `ch`, producing `S + ch`.

4.6.1 Which substrings need attention?

A newly appearing substring must end at the new position. The candidates are suffixes of the old text, including the empty suffix, followed by `ch`. Some candidates may have occurred before; those are not new string values. The construction separates newly appearing strings from those already known.

Create a state `cur` for the new whole text and its long suffixes that did not occur before. Set:

```
int cur = states.size();
states.push_back(State{});
states[cur].len = states[last].len + 1;
```

To visit the old suffixes in groups, start at `p = last` and follow suffix links. `last` represents the longest suffixes of `S`; `link[last]` represents the next shorter group, and the chain continues to state 0, the empty suffix.

```
int p = last;
while (p != -1 && !states[p].next.count(ch)) {
    states[p].next[ch] = cur;  // These extensions are new strings.
    p = states[p].link;        // Move to the next shorter suffix group.
}
```

State 0 is valid. The condition is `p != -1`, not `p != 0`. Processing the root can add a one-character string; following `link[0]` then gives -1 and ends the walk. Existing transitions are detected before their destination is used.

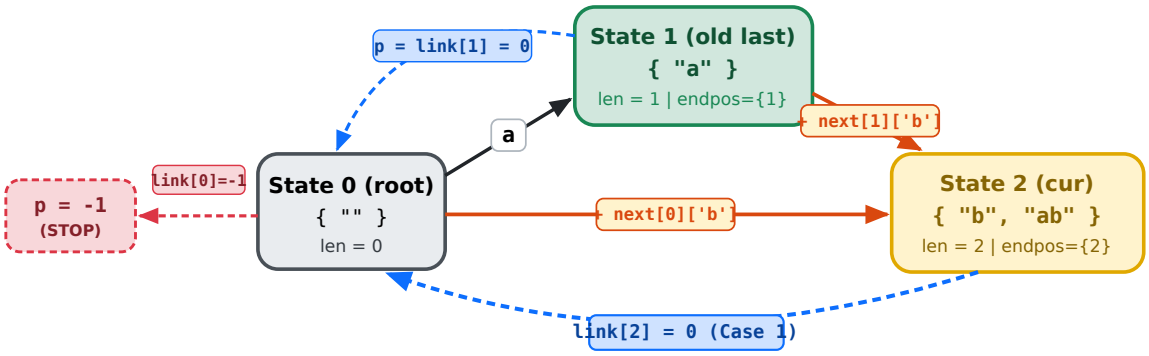
4.6.2 Three cases when the first loop stops

The updates from `a` to `ab`, and then from `ab` to `aba`, illustrate the first two cases. The next subsection develops the third case in detail.

4.6.2.1 Case 1: `p == -1`, the character has not appeared before

Start with `a` and append `b`, making `ab`:

1. Create `cur = 2` with `len[2] = 2`.
2. At `p = last = 1`, there is no `b` arrow. Add `next[1]['b'] = 2`, for `ab`, then set `p = link[1] = 0`.
3. At the root, there is no `b` arrow either. Add `next[0]['b'] = 2`, for `b`, then set `p = link[0] = -1`.
4. The loop stops at -1. No nonempty suffix of `ab` occurred in the old text, so set `link[2] = 0`.

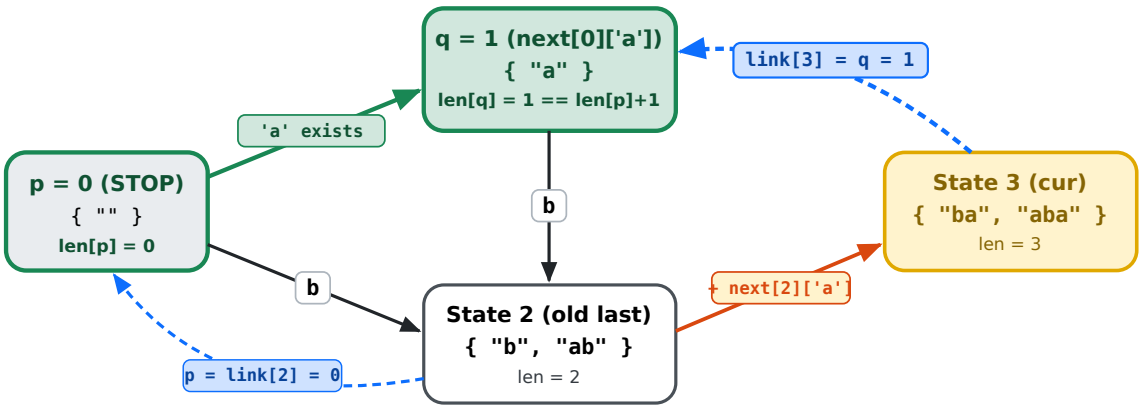


Case 1: append `b` to `a`; the walk reaches the sentinel -1.

4.6.2.2 Case 2: `len[p] + 1 == len[q]`, the existing group fits

Start with `ab` and append `a`, making `aba`:

1. Create `cur = 3` with `len[3] = 3`, for the new strings `ba` and `aba`.
2. At `p = last = 2`, representing `{b, ab}`, there is no `a` arrow. Add `next[2]['a'] = 3`, then follow `link[2]` to state 0.
3. At `p = 0`, the `a` arrow already exists and reaches `q = 1`, the state for `{a}`. Stop the loop with `p = 0`, not -1.
4. Check `len[p] + 1 = 0 + 1 = 1 = len[q]`. State `q` already has exactly the correct maximum length for the shorter suffix group. Set `link[3] = 1`.



Case 2: append `a` to `ab`; the walk stops at the root, with an existing transition.

In general, the longest previously known suffix of the new text has length `len[p] + 1`. In this equality case, it is the longest string in `q`, so all strings in that group are suffixes of the new text. The group does not need to split.

4.6.2.3 Case 3: `len[p] + 1 < len[q]`, split the group with a clone

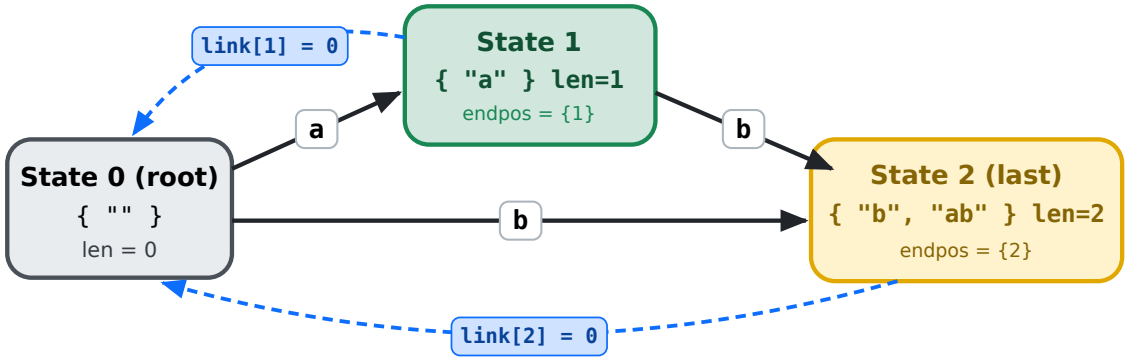
The existing destination also contains strings longer than the shorter suffix we need. Those longer strings do not end at the new position. We must separate the shorter and longer strings in that state. The next example shows exactly why, and how `len`, `next`, and `link` change.

4.7 Understand the clone: `ab` becomes `abb`

Follow three snapshots: before the new `b`, immediately after the first loop, and after resolving the conflict with clone 4.

4.7.1 Snapshot A: before the second `b`

In `ab`, both `b` and `ab` end only at position 2, so they share state 2, with `len[2] = 2`.



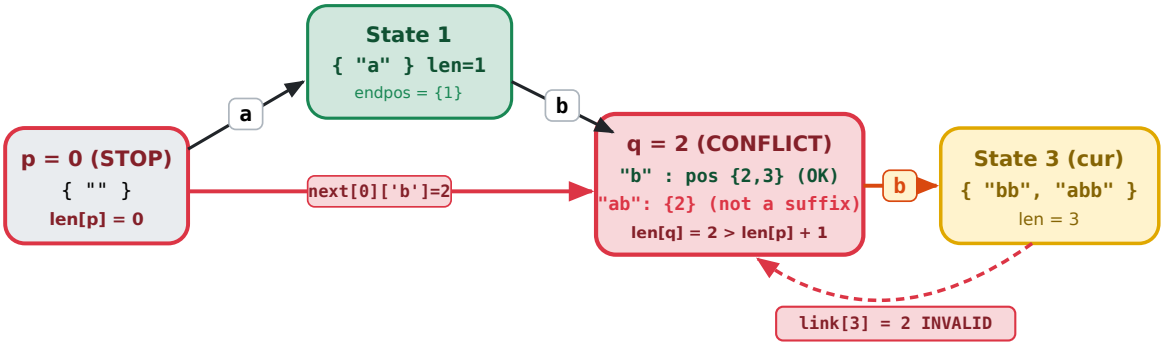
Snapshot A: `b` and `ab` share state 2 before the append.

Both paths start at state 0. The direct `b` arrow reaches state 2, and the two-character route `a`, then `b`, reaches that same state.

4.7.2 Snapshot B: the first loop exposes a conflict

Append `b` to make `abb`:

1. Create `cur = 3`, with `len[3] = 3`, for `bb` and `abb`.
2. Start at `p = 2`. There is no `b` arrow, so add `2 --b--> 3`, then set `p = link[2] = 0`.
3. At the root, `0 --b--> 2` already exists. The first loop stops, with `q = 2`.



Snapshot B: the intermediate group is inconsistent; the red proposed suffix link is invalid.

The added arrow extends two paths that already start at 0: the route for `b` can now read `bb`, and the route for `ab` can now read `abb`. Construction working at state 2 does not make it a new starting point for pattern lookups.

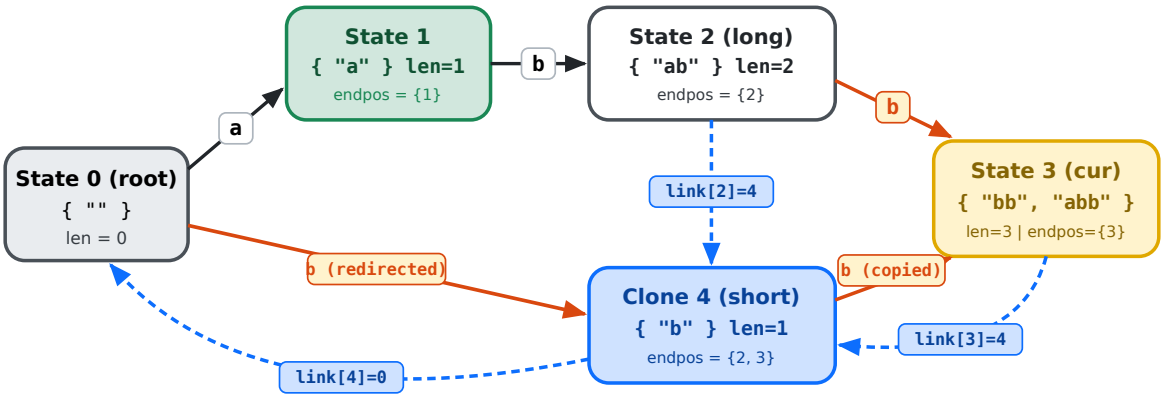
The conflict is at `q = 2`:

- Reading `b` from `p = 0` gives a suffix of length `len[p] + 1 = 1`.
- But `len[q] = 2`, because `q` also contains `ab`.
- In the new text, `b` ends at `{2, 3}`, while `ab` still ends at `{2}`. They no longer have the same `endpos`.

Setting `link[3] = 2` would point to a group containing `ab`, even though `ab` is not a suffix of `abb`. The diagram marks that proposed link as invalid; it is not a link that the algorithm actually assigns.

4.7.3 Snapshot C: separate the short and long strings

The repaired graph keeps `ab` in state 2 and puts `b` in clone 4. The copied `b` transition lets clone 4 still extend to state 3.



Snapshot C: state 2 keeps `ab`; clone 4 receives `b`.

The root's `b` route now goes through 4, so `bb` follows states 0, 4, 3. The route for `abb` still follows 0, 1, 2, 3. All lookups still start at 0.

4.7.4 The variables at the moment of the split

Variable	Value in this example	Role
`ch`	`b`	Character being appended
`cur`	3, grouping `bb, abb`	New whole-text state seeking its suffix link
`p`	0, representing the empty string	First state on the suffix chain with an existing `b` arrow
`q`	2, previously grouping `b, ab`	Existing destination, with maximum length 2 instead of the needed 1

The split leaves longer strings in `q` and transfers shorter strings to `clone`. Four operations update the fields.

4.7.4.1 Operation 1: copy `q`, then shorten the clone's maximum length

```
int clone = states.size();
State copied = states[q];          // Copy next[q] and the old link[q].
copied.len = states[p].len + 1;    // Change only the maximum length.
states.push_back(copied);
```

Why `len[clone] = len[p] + 1`? Reading `ch` after the longest string in `p` forms a string of exactly that length. Here `p = 0`, so the boundary is `0 + 1 = 1`, for `b`. Within the old group `q`, strings of length at most this boundary are suffixes of the new text and go to the clone. Longer strings, here `ab`, are not those suffixes and stay in `q`.

Why copy the outgoing transitions? The shorter strings used to live in `q`, sharing all their old ending positions and possible following characters with its longer strings. They must retain those continuations. Here the clone copies the newly added `b` arrow to state 3, preserving the occurrence of `bb`.

Why copy the old suffix link? Below the shortest string transferred to the clone, the next shorter suffix group is still the same one as before the split. In this example, `link[4]` inherits the old `link[2] = 0`.

4.7.4.2 Operation 2: redirect the incoming character transitions for the shorter strings

```
while (p != -1) {
    auto it = states[p].next.find(ch);
    // Every shorter suffix also has a ch transition.
    assert(it != states[p].next.end());
    if (it->second != q) {
        break;
    }
    it->second = clone;
    p = states[p].link;
}
```

Before the split, two `b` arrows entered state 2:

- `1 --b--> 2` extends `a` to the longer string `ab`. It must stay at `q = 2`.
- `0 --b--> 2` extends the empty string to the shorter string `b`. It must change to `0 --b--> 4`.

The code finds exactly these shorter contexts by following suffix links from `p`. While a `ch` transition still points to `q`, redirect it to the clone. Stop at -1 or at an ancestor whose transition no longer points to `q`. The other incoming arrows continue to point to the longer group in `q`.

An existing transition is guaranteed here. The first loop has already found a string `P + ch` occurring in the old text. Every shorter suffix `S + ch` also occurs there, so every suffix-link

ancestor reached during redirection has a `ch` transition. The assertion states this invariant; a missing transition would indicate a construction bug, not a normal stopping condition. This guarantee applies to the redirection loop, not to the first loop that adds missing transitions.

The destination need not remain `q`. Keep `q` fixed as `p` moves up its suffix links. Shorter strings ending in `ch` can have additional ending positions. Once a transition reaches a different group, that group has strictly more ending positions than the original group of `q`. Still shorter suffixes retain these extra positions, so none can return to `q`. This is why the single `break` stops the entire redirection search rather than skipping one ancestor.

4.7.4.3 Operation 3: make the clone the suffix-link parent of `q`

```
states[q].link = clone;
```

State `q` now contains only longer strings. Its new shortest represented length is `len[clone] + 1`. Shortening beyond that boundary enters the clone's group. Here removing the leading `a` from `ab` gives `b`, now in clone 4, so `link[2] = 4`.

4.7.4.4 Operation 4: make the clone the suffix-link parent of `cur`

```
states[cur].link = clone;
```

This was the original goal: find the group containing the longest suffix of the new text that already occurred in the old text. For `abb`, that string is `b`. Its group now exists separately in clone 4, so `link[3] = 4`.

4.7.5 Check every changed field

Field	Before the split	After the split
Strings in `q`	`b, ab`	`ab`
Strings in `clone`	No clone	`b`
Maximum lengths	`len[q] = 2`	`len[q] = 2`, `len[clone] = 1`
Clone's outgoing arrows	No clone	Copy `next[q]`, including its `b` arrow to 3
Incoming `b` from 1	Destination 2	Still 2
Incoming `b` from 0	Destination 2	Now 4
Clone's suffix link	No clone	0, the old `link[2]`
Old state's suffix link	`link[2] = 0`	`link[2] = 4`
New state's suffix link	Not assigned	`link[3] = 4`

4.7.6 Why the ending-position invariant must be preserved

In a deterministic automaton, strings reaching the same state share the same outgoing transition table. Once there, the machine cannot distinguish which of those strings we used to arrive.

Suppose `b`, ending at `{2, 3}`, and `ab`, ending only at `{2}`, were left in the same state after building `abb`. Now append `c` to make `abbc`:

- The `b` ending at position 3 can continue with `c`: `bc` exists.
- The only `ab` ends at position 2, where the next character is `b`: `abc` does not exist.

If both strings shared a transition table, adding the `c` continuation needed by `b` would incorrectly allow `abc`. Keeping equal `endpos` within each state prevents that false positive during later updates.

4.7.7 Why at most one clone is needed in an update

Appending `ch` at position `L + 1` changes old ending-position sets only for strings that are suffixes of the new text. Such strings acquire the new ending position `L + 1`. For any old state, there are three possibilities:

1. **None of its strings is a suffix of the new text.** Their ending-position sets stay unchanged, so the group stays valid.
2. **All its strings are suffixes of the new text.** Every string in the group gains the same new ending position. In the equality case this includes `q` and all of its suffix-link ancestors. They still share one `endpos`.
3. **Only its shorter strings are suffixes of the new text.** This can happen at just the boundary state `q`: the longest previously known suffix ends inside its interval of lengths. Split `q` at `len[p] + 1`. The longer strings stay in `q`, keeping their old ending positions. The shorter strings move to the clone and gain the new ending position.

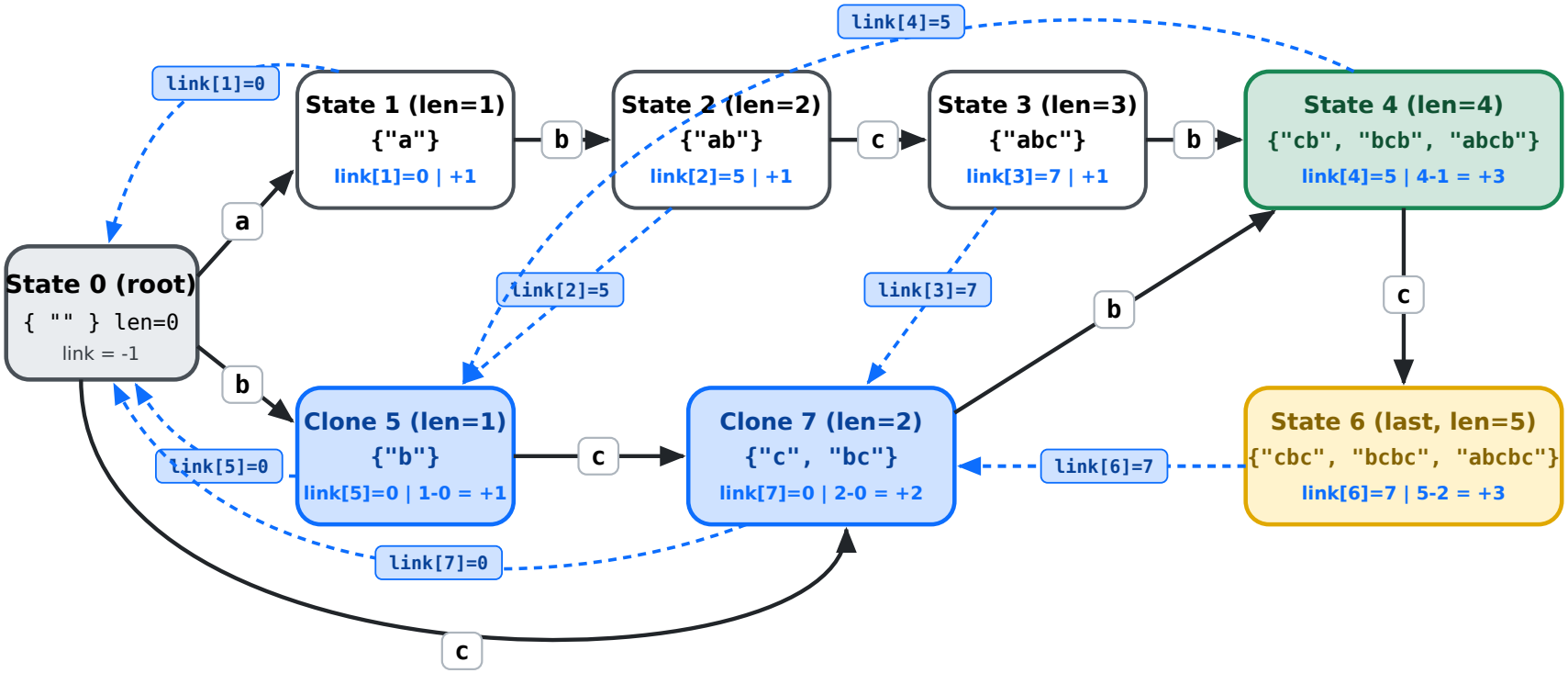
Why is there only one boundary state? Follow the previously known suffixes of the new text from longest to shortest. The longest one reaches `q`; all shorter groups reached by suffix links are included completely. An old group outside this chain contains none of those suffixes. Only the first group can be included partially.

One split therefore restores the invariant everywhere. Every append creates one whole-text state `cur` and at most one additional state `clone`.

4.8 The complete `abcbc` automaton and its suffix-link tree

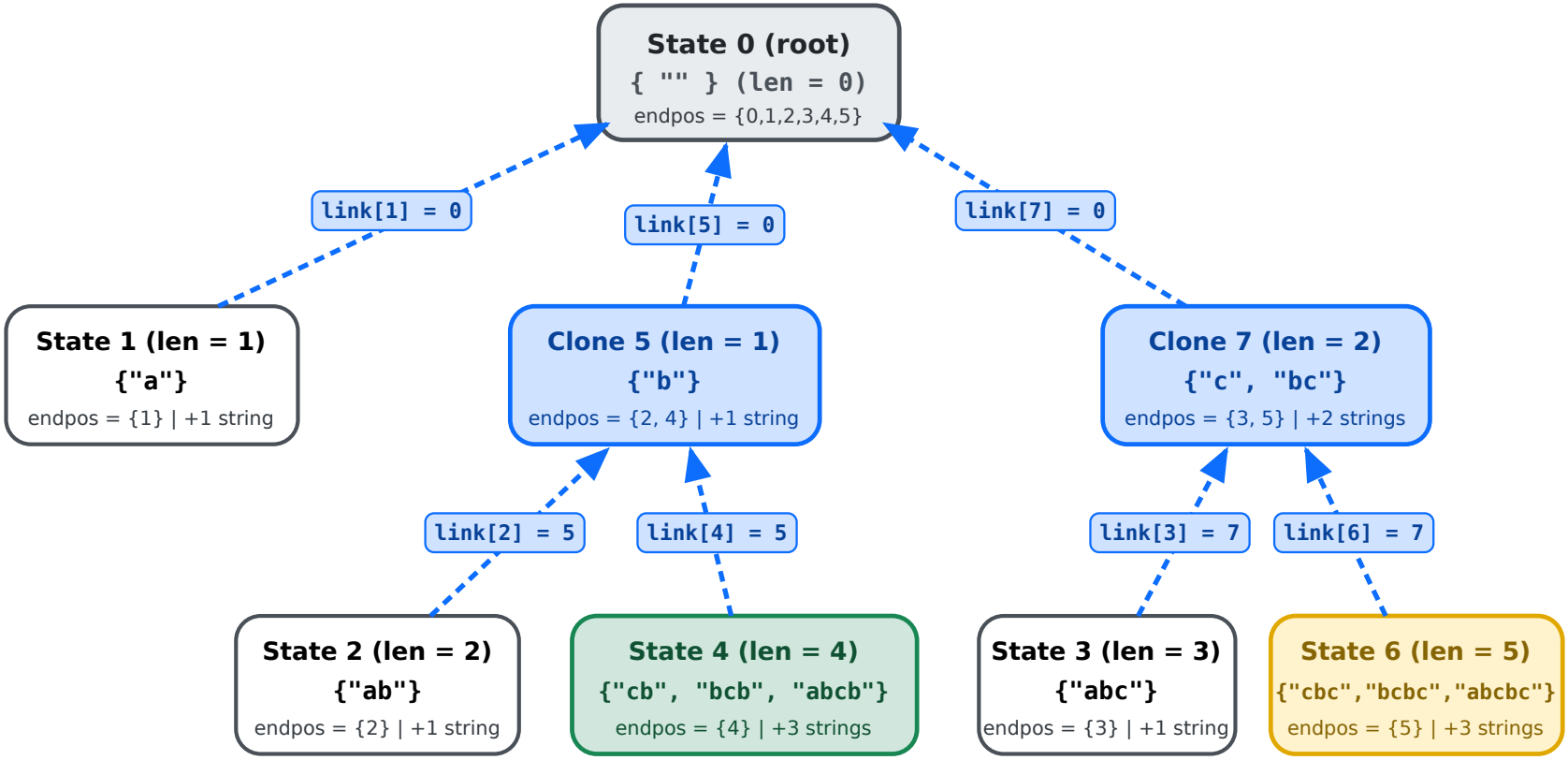
The full automaton for `abcbc` has **seven non-root states and eight states including the root**. The black arrows read characters; dashed blue arrows are suffix links. Each box lists its strings and its contribution to the distinct-substring count.

Complete `abcbc` automaton: seven non-root states, plus the root.



Because character transitions and suffix links cross in the full graph, now isolate just the suffix links. We obtain the **suffix-link tree**, rooted at state 0, with arrows from each state toward its suffix-link parent.

Suffix-link tree for `abcbc`; arrows point toward the shorter suffix group.



Moving toward the root removes enough leading characters to reach a string with additional ending positions. For the empty string, we use all text boundaries `{0, 1, 2, 3, 4, 5}` as ending positions; state 0 is excluded from the count of nonempty strings.

4.8.1 Explain every suffix link in `abcbc`

Remove one character from the left of the *shortest* string in a state's group. The result belongs to its suffix-link parent:

State	Its strings	Shortest string, shortened	Link destination
1	`a`	`a` becomes empty	0
2	`ab`	`ab` becomes `b`	5
3	`abc`	`abc` becomes `bc`	7
4	`cb, bcb, abcb`	`cb` becomes `b`	5
5	`b`	`b` becomes empty	0
6	`cbc, bcbc, abcbc`	`cbc` becomes `bc`	7
7	`c, bc`	`c` becomes empty	0

Now count each state's strings using maximum lengths, not state identifiers:

State	`len[v]`	`len[link[v]]`	Contribution
1	1	0	1
2	2	1	1
3	3	2	1
4	4	1	3
5	1	0	1
6	5	2	3
7	2	0	2

The sum is `1 + 1 + 1 + 3 + 1 + 3 + 2 = 12` distinct nonempty substrings. For example, `link[6] = 7`, but its contribution subtracts `len[7] = 2`, not the identifier 7.

4.9 Commented C++ and Python implementations

The fields directly encode the groups discussed above:

- `next[v][ch]` is a character transition: append `ch` to strings in state `v`.
- `link[v]` points to the next shorter suffix group.
- `len[v]` is the longest represented length; the Python version calls it `length[v]`.
- `last` is the state for the whole text built so far. A clone can have a larger identifier than `last`.

The reusable classes below provide `extend`, `contains`, and `count_distinct`. All lookups start at 0. They accept the empty pattern; the empty automaton has zero distinct nonempty substrings. Appending is supported; deleting or replacing earlier characters is not.

4.9.1 C++

```cpp
#pragma once

#include <bits/stdc++.h>
using namespace std;

class SuffixAutomaton {
    struct State {
        map<char, int> next;  // Character transitions: extend to the right.
        int link = -1;        // Next group of shorter suffixes.
        int len = 0;          // Longest string in this group.
    };
    vector<State> states{State{}};
    int last = 0;

public:
    bool contains(const string& pattern) const {
        int v = 0;
        for (char ch : pattern) {
            auto it = states[v].next.find(ch);
            if (it == states[v].next.end()) {
                return false;
            }
            v = it->second;
        }
        return true;
    }

    void extend(char ch) {
        // 1. Make a state for the new whole text and its new suffixes.
        int cur = states.size();
        states.push_back(State{});
        states[cur].len = states[last].len + 1;
        int p = last;
```

```
        // 2. Visit old suffix groups that have no ch continuation.
        while (p != -1 && !states[p].next.count(ch)) {
            states[p].next[ch] = cur;
            p = states[p].link;
        }

        // 3. Find the next shorter group for cur.
        if (p == -1) {
            // Case 1: ch is new; only the empty suffix was known.
            states[cur].link = 0;
        } else {
            int q = states[p].next.at(ch);
            if (states[p].len + 1 == states[q].len) {
                // Case 2: all of q is the next suffix group.
                states[cur].link = q;
            } else {
                // Case 3: split q; clone keeps its shorter strings.
                int clone = states.size();
                State copied = states[q];
                copied.len = states[p].len + 1;
                states.push_back(copied);

                // Redirect only the shorter incoming contexts.
                while (p != -1) {
                    auto it = states[p].next.find(ch);
                    // Every shorter suffix also has a ch transition.
                    assert(it != states[p].next.end());
                    if (it->second != q) {
                        break;
                    }
                    it->second = clone;
                    p = states[p].link;
                }
                // Both longer groups now have clone as suffix parent.
                states[q].link = clone;
                states[cur].link = clone;
            }
        }
        last = cur;
    }

    long long count_distinct() const {
        long long answer = 0;
        for (int v = 1; v < (int)states.size(); v++) {
            int parent = states[v].link;
            answer += states[v].len - states[parent].len;
        }
        return answer;
    }
};
```

The transition maps cost `O(log alphabet)` per lookup or update. Building the text takes `O(n log alphabet)` amortized time and `O(n)` memory for a fixed alphabet. Reading a pattern of length `m` takes `O(m log alphabet)` time. `count_distinct()` scans the states and costs `O(n)` on each call. Calling it after every append of a large text can therefore take quadratic time.

Copy the clone before growing the state vector, and retain state indices rather than references that vector growth could invalidate. The substring count can reach `n * (n + 1) / 2`, so it needs `long long`.

4.9.2 Python

```python
class SuffixAutomaton:
    def __init__(self):
        self.next = [{}]
        self.link = [-1]
        self.length = [0]
        self.last = 0

    def contains(self, pattern):
        v = 0
        for ch in pattern:
            if ch not in self.next[v]:
                return False
            v = self.next[v][ch]
        return True

    def extend(self, ch):
        # 1. Make a state for the new whole text and its new suffixes.
        cur = len(self.next)
        self.next.append({})
        self.length.append(self.length[self.last] + 1)
        self.link.append(0)
        p = self.last

        # 2. Visit old suffix groups that have no ch continuation.
        while p != -1 and ch not in self.next[p]:
            self.next[p][ch] = cur
            p = self.link[p]

        # 3. Find the next shorter group for cur.
        if p == -1:
            # Case 1: ch is new; only the empty suffix was known.
            self.link[cur] = 0
        else:
            q = self.next[p][ch]
            if self.length[p] + 1 == self.length[q]:
                # Case 2: all of q is the next suffix group.
                self.link[cur] = q
            else:
                # Case 3: split q; clone keeps its shorter strings.
```

```
                # Copy the dictionary so the two states are independent.
                clone = len(self.next)
                self.next.append(self.next[q].copy())
                self.length.append(self.length[p] + 1)
                self.link.append(self.link[q])

                # Redirect only the shorter incoming contexts.
                while p != -1:
                    # Every shorter suffix also has a ch transition.
                    assert ch in self.next[p]
                    if self.next[p][ch] != q:
                        break
                    self.next[p][ch] = clone
                    p = self.link[p]
                # Both longer groups now have clone as suffix parent.
                self.link[q] = clone
                self.link[cur] = clone

        self.last = cur

    def count_distinct(self):
        return sum(
            self.length[v] - self.length[self.link[v]]
            for v in range(1, len(self.next))
        )
```

Dictionaries give expected constant-time transition operations. For the fixed alphabet, building takes expected `O(n)` time and `O(n)` memory. Pattern lookup takes expected `O(m)` time, and `count_distinct()` takes `O(n)` time. Each node must own its own dictionary: use `.copy()` when cloning. Python integers grow automatically to hold the count.

4.10 Quick reference and checks

1. **State `v`:** a group of substrings with exactly the same ending-position set in the indexed text.
2. **Strings in `v`:** one suffix of each length from `len[link[v]] + 1` through `len[v]`, inclusive.
3. **`link[v]`:** shorten the longest string from the left until it acquires additional ending positions; the first different group is the destination.
4. **Clone:** split a state when only its shorter strings acquire the new ending position. Keep the longer strings in `q` and move the shorter strings into `clone`.
5. **Character transition:** read a character and filter the possible occurrences. It is enough that one occurrence can continue.
6. **Membership:** start at 0 and follow only character transitions. Suffix links guide construction and delimit represented length intervals.

For small texts, enumerate all substrings into a set and compare its size with `count_distinct()`. Compare `contains(pattern)` with direct substring search. Repeat after every append, including appends that create clones. Useful checks are `aba` (5 strings), `abb` (5), `abcbc` (12), and `abcbcd` (18). Also test `aaaa` (4), `abcd` (10), and the empty text (0).

Check the distinction from the guide directly: in `abcbc`, both `c` and `bc` reach state 7, whose possible endings are `{3, 5}`. Reading another `b` reaches state 4, with only ending position `{4}`. Appending `d` later adds a different continuation from state 7, supplied by its occurrence at position 5.

5 Implementation notes and tests

Tries in C++. For lowercase input, `vector<array<int,26>>` with node `0` as the root and `0` meaning "absent" is compact and fast. Reserve capacity for the total input length; that is the exact upper bound on node count.

Tries in Python. A `dict` per node is clearer and usually faster than a 26-slot list for sparse alphabets. Check for a missing edge before allocating a new child dictionary, as in the implementation above.

Everything else. Read input with `sys.stdin.buffer.read()`. For hashing, use two independent moduli or a random base — a single fixed 32-bit modulus is attackable. Palindrome endpoints arrive 1-based and inclusive; convert once, at the point of reading, not repeatedly inside the loop.

Case	What it catches
a word that is a prefix of another word (`app`, `apple`)	treating `cnt` as an end-of-word counter
duplicate words in the dictionary	a unique-prefix answer that must be `-1`
query prefix absent from the trie	a missing-edge path that walks past the end
single-character palindrome query	the `l < r` loop bound
even-length palindrome (`abba`)	an off-by-one that only breaks one parity
odd-length palindrome (`aba`)	the same off-by-one, the other parity
all-equal string (`aaaa`)	the maximal-state-sharing case
one-character string	an empty contribution sum

Useful values to check by hand: the trie over `app, apple, ape, bat` gives `cnt["ap"] = 3` and `cnt["app"] = 2`; `abacaba` answers NO for `[2,4]` and YES for `[3,5]`; and `aba` has exactly `5` distinct non-empty substrings.

Run the local reference suite with:

```
CP_TARGET=solution python3 sections/44_string_algorithms_ii/check.py
```

6 Exercises

Use A to fix the trie invariant before writing anything: `cnt[v]` counts words passing through `v`, not words ending at `v`. Use B to convert the 1-based inclusive endpoints exactly once and to test both parities. Use C to state the `len[v] - len[link[v]]` invariant and use the automaton at its large input bound; a set of substrings is useful only as a tiny-input testing oracle. Use D as the blind synthesis problem: the shortest unique prefix of a word is the first node on its path whose count has fallen to one, so decide what the trie must store and what it means when the count never falls. Test a word that is a prefix of another, duplicate words, a prefix that is absent, and single-character inputs.

- A. Trie Prefix Count: Given dictionary words and prefix queries, count words having each prefix.
- B. Palindrome Queries: For each substring query, print whether the substring is a palindrome.
- C. Distinct Substrings: Print the number of distinct non-empty substrings of a string.
- D. Unique Prefix Lengths: For each word, print the length of its shortest prefix shared by exactly one input word, or `-1`.